SC1001: Introduction to Computational Thinking

Week 9: Functions

Jordan Boyd-Graber

Nanyang Technological University

Slides

Slides and updates will be posted at

`boydgraber.org/teaching/SC_1001/`

A Quick Aside

I need you to answer questions so I can gauge the pace and content.

- If a question feels too simple, answer it anyway.
- Many quick correct answers tell me we can move faster.
- If most people are comfortable, I will add more advanced material.
- If a question feels too hard, answer the smaller follow-up questions.
- Those smaller questions help us move through the problem step by step.

Overview

1. Read the rules for the 1A2B game
2. Design the game as a sequence of steps
3. Turn those steps into function prototypes
4. Implement each function
5. Combine the functions into a complete program

Bulls & Cows

We will build a Python version of 1A2B, also called Bulls & Cows.

- The program randomly chooses four distinct digits as the secret number.
- In each round, the player guesses four digits.
- A digit earns `1A` if it appears in the secret and is in the same position.
- A digit earns `1B` if it appears in the secret but is in a different position.
- The player wins when a guess earns `4A`.

A Possible Game Transcript

```
Please input 4 digits: 1234
0 A (Bulls) 2 B (Cows)
```

`0A 2B` means two digits are correct, but both are in wrong positions.

Possible secrets include:

`24??` `2?4?` `4?2?` `4??2`

A Possible Game Transcript

```
Please input 4 digits: 1234
0 A (Bulls) 2 B (Cows)
Please input 4 digits: 2478
3 A (Bulls) 0 B (Cows)
```

3A 0B means three positions are correct and the fourth guessed digit is not in the secret.

Possible secrets include:

?478 2?78 24?8 247?

A Possible Game Transcript

```
Please input 4 digits: 1234
0 A (Bulls) 2 B (Cows)
Please input 4 digits: 2478
3 A (Bulls) 0 B (Cows)
Please input 4 digits: 2468
You Win!!
Attempts:  3
```

2468 gives 4A, so the secret is exactly 2468.

The main loop can stop after printing the number of attempts.

Discussion 1: Game Design

To play the 1A2B game, what steps does the program need?

- What should happen before the first guess?
- What should happen during each round?
- What information must the program remember?
- When should the loop stop?

Write the design first as pseudocode or a flowchart.

Discussion 1: One Algorithm

1. Generate the secret number with four distinct digits.

Discussion 1: One Algorithm

1. Generate the secret number with four distinct digits.
2. Ask the user for a four-digit guess.

Discussion 1: One Algorithm

1. Generate the secret number with four distinct digits.
2. Ask the user for a four-digit guess.
3. Compare the guess with the secret number.

Discussion 1: One Algorithm

1. Generate the secret number with four distinct digits.
2. Ask the user for a four-digit guess.
3. Compare the guess with the secret number.
4. Print the number of bulls and cows.

Discussion 1: One Algorithm

1. Generate the secret number with four distinct digits.
2. Ask the user for a four-digit guess.
3. Compare the guess with the secret number.
4. Print the number of bulls and cows.
5. If the guess is not `4A`, return to step 2.

Discussion 1: One Algorithm

1. Generate the secret number with four distinct digits.
2. Ask the user for a four-digit guess.
3. Compare the guess with the secret number.
4. Print the number of bulls and cows.
5. If the guess is not `4A`, return to step 2.
6. If the guess is `4A`, end the game.

Discussion 2: Function Interfaces

Based on the algorithm, define a function for each step before implementing the body.

- What does the function do?
- How many parameters does it need?
- What should each parameter represent?
- What should the function return?

Generate the Secret

```
def gen_secret(allowed_digits: list[str], count: int) -> list[str]:
    """Returns distinct random digits from a list.

    Args:
        allowed_digits: A list of digits that may appear in the secret.
        count: The number of digits to generate.

    Returns:
        A list containing count distinct digits.
    """
```

Read the User Guess

```
def user_guess(allowed_digits: list[str], count: int) -> list[str]:
    """Reads one valid guess of specified length from the user.

    Args:
        allowed_digits: A list of digits that may appear in the secret.
        count: The number of digits to generate.

    Returns:
        A list containing the four guessed digits.
    """
```

Compare Guess and Secret

```
def check_guess(guess: list[str], secret: list[str]) -> tuple[int, int]:
    """Counts bulls and cows for one guess.

    Args:
        guess: The digits guessed by the user.
        secret: The hidden digits.

    Returns:
        A tuple with the number of bulls and cows.
    """
```

Discussion 3: Implement the Functions

Implement each function according to the interface.

- Use `random.shuffle()` to generate the secret.
- Alternative: use `random.sample()` to select digits.
- Check each character against the allowed digit list.
- Return `"1234"` as `["1", "2", "3", "4"]`.
- Compare every guessed digit with every secret digit.

```
def gen_secret(allowed_digits: list[str], count: int) -> list[str]:

def user_guess(allowed_digits: list[str], count: int) -> list[str]:

def check_guess(guess: list[str], secret: list[str]) -> tuple[int, int
```

Constants and Setup

```
import random

WIDTH = 4
ALLOWED_DIGITS = [str(digit) for digit in range(10)]
```

`WIDTH` makes the four-digit rule explicit and reusable.

Implement `gen_numbers`

```
def gen_secret(allowed_digits: list[str], count: int) -> list[str]:
```

Start with the signature. The function needs the available digits and the number to select.

Implement gen_numbers

```
def gen_secret(allowed_digits: list[str], count: int) -> list[str]:
    random.shuffle(allowed_digits)
```

Shuffle the list so the digits appear in a random order.

Implement gen_numbers

```
def gen_secret(allowed_digits: list[str], count: int) -> list[str]:
    random.shuffle(allowed_digits)
    numbers = []
```

Create an empty result list for the selected digits.

Implement gen_numbers

```
def gen_secret(allowed_digits: list[str], count: int) -> list[str]:
    random.shuffle(allowed_digits)
    numbers = []

    for i in range(count):
        numbers.append(allowed_digits[i])
```

Copy the first count shuffled digits into the result list.

Implement gen_numbers

```
def gen_secret(allowed_digits: list[str], count: int) -> list[str]:
    random.shuffle(allowed_digits)
    numbers = []

    for i in range(count):
        numbers.append(allowed_digits[i])

    return numbers
```

Return the generated secret so the main program can use it.

Alternative `gen_numbers`

Python can select several distinct items directly.

```
def gen_secret(allowed_digits: list[str], count: int) -> list[str]:
    return random.sample(allowed_digits, count)
```

`random.sample()` returns a new list, so it does not shuffle the original `allowed_digits`.

Implement `user_guess`

```
def user_guess(allowed_digits: list[str], count: int) -> list[str]:
```

The allowed digit list is explicit, so validation does not rely on string methods.

Implement `user_guess`

```
def user_guess(allowed_digits: list[str], count: int) -> list[str]:
    input_str = input(f"Please input {count} digits: ")
```

Read the user's guess as a string. Input from `input()` is always text.

Implement user_guess

```
def user_guess(allowed_digits: list[str], count: int) -> list[str]:
    input_str = input("Please input 4 digits: ")
    while (
        len(input_str) != WIDTH
        or not all(digit in allowed_digits for digit in input_str)
    ):
        input_str = input(f"Wrong format!! Please input {count} digits: ")
```

Repeat the prompt unless the input has the right length and every character is allowed.

Implement user_guess

```
def user_guess(allowed_digits: list[str], count: int) -> list[str]:
    input_str = input(f"Please input {count} digits: ")
    while (
        len(input_str) != count
        or not all(digit in allowed_digits for digit in input_str)
    ):
        input_str = input(f"Wrong format!! Please input {count} digits: ")

    guess = []
    for digit in input_str:
        guess.append(digit)
```

After validation, append each character to the guess list.

Implement user_guess

```
def user_guess(allowed_digits: list[str], count: int) -> list[str]:
    input_str = input(f"Please input {count} digits: ")
    while (
        len(input_str) != count
        or not all(digit in allowed_digits for digit in input_str)
    ):
        input_str = input(f"Wrong format!! Please input {count} digits: ")

    guess = []
    for digit in input_str:
        guess.append(digit)

    return guess
```

Return the list so the main loop can compare it with the secret.

Alternative `user_guess`

The loop that builds the list can also be written as a list comprehension.

```
def user_guess(allowed_digits: list[str], count: int) -> list[str]:
    input_str = input(f"Please input {count} digits: ")
    while (
        len(input_str) != count
        or not all(digit in allowed_digits for digit in input_str)
    ):
        input_str = input(f"Wrong format!! Please input {count} digits: ")

    return [digit for digit in input_str]
```

Input Validation

The validation checks two things:

- `len(input_str) != count` checks whether the user entered four characters.
- `digit in allowed_digits` checks whether each character is allowed.

What other rule might you add?

Input Validation

The validation checks two things:

- `len(input_str) != count` checks whether the user entered four characters.
- `digit in allowed_digits` checks whether each character is allowed.

What other rule might you add? Prevent repeated (valid) items

Implement `check_guess`

```
def check_guess(guess: list[str], secret: list[str]) -> tuple[int, int]:
```

The comparison needs both lists: the guess and the secret.

Implement `check_guess`

```
def check_guess(guess: list[str], secret: list[str]) -> tuple[int, int]:
    bulls = 0
    cows = 0
```

Start both counters at zero before checking any digits.

Implement `check_guess`

```
def check_guess(guess: list[str], secret: list[str]) -> tuple[int, int]:
    bulls = 0
    cows = 0

    for guess_position, guess_digit in enumerate(guess):
        for secret_position, secret_digit in enumerate(secret):
```

Use `enumerate` so each loop has both a position and a digit.

Implement check_guess

```
def check_guess(guess: list[str], secret: list[str]) -> tuple[int, int]:
    bulls = 0
    cows = 0

    for guess_position, guess_digit in enumerate(guess):
        for secret_position, secret_digit in enumerate(secret):
            if guess_digit == secret_digit:
                if guess_position == secret_position:
                    bulls += 1
                else:
                    cows += 1
```

A matching digit in the same position is a bull. A matching digit in a different position is a cow.

Implement check_guess

```
def check_guess(guess: list[str], secret: list[str]) -> tuple[int, int]:
    bulls = 0
    cows = 0

    for guess_position, guess_digit in enumerate(guess):
        for secret_position, secret_digit in enumerate(secret):
            if guess_digit == secret_digit:
                if guess_position == secret_position:
                    bulls += 1
                else:
                    cows += 1

    return (bulls, cows)
```

Return both counts together as one tuple.

Alternative check_guess

Another version can ask whether each secret digit appears anywhere in the guess.

```
def check_guess(guess: list[str], secret: list[str]) -> tuple[int, int]:
    bulls = 0
    cows = 0

    for position, digit in enumerate(secret):
        if digit == guess[position]:
            bulls += 1
        elif digit in guess:
            cows += 1

    return (bulls, cows)
```

```
def check_guess(guess: list[str], secret: list[str]) -> tuple[int, int]:
    bulls = [x == y for x, y in zip(guess, secret)]
    cows = [guess_content in secret for pos, guess_content in enumerate(guess) if not bulls[pos]]

    return (sum(int(x) for x in bulls), sum(int(x) for x in cows))
```

Why Not `continue`?

Repeated digits need careful matching

```
guess = "1123"
secret = "0111"
```

The result should be `1A 2B`, since the last two 1s in the secret are both the correct digit in the wrong location.

Before adding shortcuts, decide how each copy of a repeated digit gets matched.

Discussion 4: Combine the Functions

Use the functions to complete the game.

- Generate the secret once.
- Repeatedly read and check guesses.
- Count the number of attempts.
- Stop when the result has four bulls.

Main Program

```
secret = gen_secret(ALLOWED_DIGITS, WIDTH)
attempts = 0
```

Generate one hidden secret and start the attempt counter.

Main Program

```
secret = gen_secret(ALLOWED_DIGITS, WIDTH)
attempts = 0

while True:
    guess = user_guess(ALLOWED_DIGITS)
    attempts += 1
```

Each loop iteration reads one guess and counts it as an attempt.

Main Program

```
secret = gen_secret(ALLOWED_DIGITS, WIDTH)
attempts = 0

while True:
    guess = user_guess(ALLOWED_DIGITS)
    attempts += 1

    result = check_guess(guess, secret)
```

The helper function turns the guess and secret into a result tuple.

Main Program

```
secret = gen_secret(ALLOWED_DIGITS, WIDTH)
attempts = 0

while True:
    guess = user_guess(ALLOWED_DIGITS)
    attempts += 1

    result = check_guess(guess, secret)

    if result[0] == 4:
        print("You Win!!")
        print("Attempts: ", attempts)
        break
```

If the result has four bulls, the player has won and the loop stops.

Main Program

```
secret = gen_secret(ALLOWED_DIGITS, WIDTH)
attempts = 0

while True:
    guess = user_guess(ALLOWED_DIGITS)
    attempts += 1

    result = check_guess(guess, secret)

    if result[0] == 4:
        print("You Win!!")
        print("Attempts: ", attempts)
        break
    else:
        print(result[0], "A (Bulls)", result[1], "B (Cows)")
```

Otherwise, print feedback and let the loop ask for another guess.

Dumb Way to Play 1: Try Everyting

- How many possible solutions are there?

Dumb Way to Play 1: Try Everyting

- How many possible solutions are there?

$$10 \cdot 9 \cdot 8 \cot 7 = \frac{10!}{6!} = 5040 \tag{1}$$

Dumb Way to Play 1: Try Everyting

- How many possible solutions are there?

$$10 \cdot 9 \cdot 8 \cot 7 = \frac{10!}{6!} = 5040 \tag{1}$$

- Just try them one by one

Dumb Way to Play 2: Probe

- Guess 1111, 2222, etc.

Dumb Way to Play 2: Probe

- Guess 1111, 2222, etc.
- You'll learn which digits are in the secret

Dumb Way to Play 2: Probe

- Guess 1111, 2222, etc.
- You'll learn which digits are in the secret
- Then try all $4 \cdot 3 \cdot 2 = 24$ orderings

Dumb Ways to Play the Game

Strategy	Worst Case	Average Case
Exhaustive	5040	2520

Dumb Ways to Play the Game

Strategy	Worst Case	Average Case
Exhaustive	5040	2520
Probe	34	17

Discussion 5: Solver Pseudocode

Create pseudocode for an automated solver.

- Generate all possible solutions.
- Make a guess and receive the `A/B` result.
- Remove impossible solutions.
- Generate the next guess

Solver Pseudocode

1. Generate the set of all possible solutions.
2. Start with a first guess.
3. Check the guess against the hidden secret.
4. If all four digits are bulls, stop.
5. Remove any solution that would not produce the same result.
6. Choose the next guess from the remaining solutions.
7. Repeat from step 3.

The pseudocode separates two ideas: using feedback to shrink the search space, and deciding which remaining secret to try next.

Discussion 6: Solver Code

Write code for the solver pieces.

- Remove excluded solutions after each result.
- Pick a possible solution as the next guess.
- Run the solver many times and measure wall-clock time.

Solver Type Hints

```
from collections.abc import Callable

Guess = tuple[int, ...]
NextGuess = Callable[[set[Guess], set[Guess]], Guess]

def solve(secret: Guess, generate_next_guess: NextGuess) -> int:
```

`NextGuess` names the kind of function the solver can call.

Removing Incompatible Solutions

```
possible_solutions = set(ALL_SOLUTIONS)
guesses = set()
steps = 0
guess = (1, 2, 3, 4)
```

Start with every possible no-repeat secret, a record of guesses, and a fixed first guess.

Removing Incompatible Solutions

```
possible_solutions = set(ALL_SOLUTIONS)
guesses = set()
steps = 0
guess = (1, 2, 3, 4)

while True:
    steps += 1
    guesses.add(guess)
    result = check_guess(guess, secret)
```

Record the guess, then use `check_guess` to get feedback for the real secret.

Removing Incompatible Solutions

```
possible_solutions = set(ALL_SOLUTIONS)
guesses = set()
steps = 0
guess = (1, 2, 3, 4)

while True:
    steps += 1
    guesses.add(guess)
    result = check_guess(guess, secret)

    if result[0] == WIDTH:
        return steps
```

A result with four bulls means the solver has found the secret.

Removing Incompatible Solutions

```
possible_solutions = set(ALL_SOLUTIONS)
guesses = set()
steps = 0
guess = (1, 2, 3, 4)

while True:
    steps += 1
    guesses.add(guess)
    result = check_guess(guess, secret)

    if result[0] == WIDTH:
        return steps

    for possible_secret in tuple(possible_solutions):
        if check_guess(guess, possible_secret) != result:
            possible_solutions.remove(possible_secret)
```

Loop over a copy of the set so secrets can be removed while checking.

Removing Incompatible Solutions

```
possible_solutions = set(ALL_SOLUTIONS)
guesses = set()
steps = 0
guess = (1, 2, 3, 4)

while True:
    steps += 1
    guesses.add(guess)
    result = check_guess(guess, secret)

    if result[0] == WIDTH:
        return steps

    for possible_secret in tuple(possible_solutions):
        if check_guess(guess, possible_secret) != result:
            possible_solutions.remove(possible_secret)

    guess = generate_next_guess(possible_solutions, guesses)
```

After filtering, delegate the next-guess decision to another function.

A Simple Next Guess

```
def random_guess(solutions: set[Guess], guesses: set[Guess]) -> Guess:
```

This function receives only the remaining possible solutions.

A Simple Next Guess

```
def random_guess(solutions: set[Guess], guesses: set[Guess]) -> Guess:
    return random.choice(tuple(solutions))
```

Convert the set to a tuple because `random.choice` needs a sequence.

Discussion 7: Guess Quality

What makes a good guess?

- A good guess should be informative.
- It should exclude many options.
- One baseline scans all 5040 no-repeat guesses.
- Another baseline tries 0000, 1111, and so on.
- Compare average steps and worst case across all secrets.
- More advanced versions use entropy or game-tree search.
- Today: count how many possible outcomes each guess can produce.

Most Diverse Guess

```
def diverse_guess(solutions: set[Guess], guesses: set[Guess]) -> Guess:
    best_guess = None
    best_score = -1
```

Track the best guess found so far and the number of different responses it can produce.

Most Diverse Guess

```
def diverse_guess(solutions: set[Guess], guesses: set[Guess]) -> Guess:
    best_guess = None
    best_score = -1

    for guess in solutions:
        results = set()
```

For each candidate guess, collect the responses it creates.

Most Diverse Guess

```
def diverse_guess(solutions: set[Guess], guesses: set[Guess]) -> Guess:
    best_guess = None
    best_score = -1

    for guess in solutions:
        results = set()

        for secret in solutions:
            results.add(check_guess(guess, secret))
```

Reuse `check_guess`; the heuristic should not duplicate the game rules.

Most Diverse Guess

```
def diverse_guess(solutions: set[Guess], guesses: set[Guess]) -> Guess:
    best_guess = None
    best_score = -1

    for guess in solutions:
        results = set()

        for secret in solutions:
            results.add(check_guess(guess, secret))

        if len(results) > best_score:
            best_guess = guess
            best_score = len(results)

    return best_guess
```

Choose the guess that creates the most response groups.

Testing Result

Strategy	Avg. steps	Worst case
All possible guesses	2520	5040
Repeated digit guesses	13.86	17
Random possible	5.48	8
Most diverse	5.35	8

- Scanning compatible guesses is simple and competitive.
- Repeated-digit probes answer a narrow question, so they take many steps.
- Most diverse was slightly better in the sample, but much slower.

Takeaways

- Functions work together as a team
- Arguments and return values need to be carefully considered
- Function prototypes separate design decisions from implementation details.
- Each helper function owns one part of the game.
- Return values let the main loop combine the helpers.
- The full program follows the same structure as the pseudocode.